No-Opponent-Cycle Propagators for Solving Parity Games

Gonzalo Hernandez[1,3], Julian Garcia[1], Julian Gutierrez[2], and Guido Tack[1]

[1] Monash University, Australia
{gonzalo.hernandez,julian.garcia,guido.tack}@monash.edu
[2] University of Sussex, United Kingdom {J.Gutierrez}@sussex.ac.uk
[3] Universidad de Nariño, Colombia {gonzalo.hernandez}@udenar.edu.co

Abstract. Solving parity games is a core problem in formal verification, specifically when using model checking and synthesis methods, which have several industrial-scale applications. In this paper, we propose a constraint-based approach for parity games, built upon a new propagation algorithm that eliminates opponent cycles from the game graph. This approach results in an efficient method that exploits the duality of players in a parity game. Our implementation within a Lazy Clause Generation framework outperforms all existing constraint-based methods for parity games and performs competitively against the most specialized non-CP-based algorithms—often matching and sometimes exceeding their performance—while retaining the flexibility inherent to general constraint-based approaches. Since new constraints can be easily added, our method provides a flexible framework for refining existing problem formulations to search for preferred solutions, by satisfying additional constraints. This feature can also be used to solve parity games where additional quantitative constraints (*e.g.*, cost or resource-bounded requirements) need to be satisfied. We present the theoretical foundations of our approach, an experimental evaluation, and a discussion of the results, including an analysis of different versions of the new propagator.

Keywords: Constraint programming · Parity games · Verification.

1 Introduction

Parity games are two-player zero-sum infinite-duration games played on a labelled directed graph, where vertices represent states of the players in the game and edges represent their possible available actions at every state. Solving these games is central to several techniques in automated formal verification, in particular, when solving model checking and automated synthesis problems.

Since parity games are zero-sum, perfect-information games, solving them amounts to finding which player (usually called Player Even, or ○, and Player Odd, or □) has a winning strategy, from a given starting vertex in the graph. It is known that deciding the winner in parity games is an $\mathsf{NP} \cap \mathsf{co\text{-}NP}$ problem [21], but it is not known whether or not they can be solved in polynomial time; a

problem that has remained open for nearly 50 years. Due to the importance of parity games in formal verification, several algorithms have been developed [1,3,20,22,31], all of them suffering from exponential running time limitations for specific classes of games. One such approach involves reducing the solution of a parity game to the solution of a SAT instance representing the game. The generated SAT instance is satisfiable if and only if Player Even has a winning strategy in the original game. This idea has a lot of potential given the advances in SAT-solving technology, but, as we will show later, it suffers from poor scalability due to the large size of the generated CNF formula for a given game.

In this paper, we address these scalability issues by developing a specialised propagator that takes into account the structure of parity games, resulting in significant improvements in performance and modelling power. In particular, we show that our method outperforms all other state-of-the-art constraint-based solvers. Specifically, we propose replacing the SAT-based encoding used to forbid cycles corresponding to Player Odd wins in the game graph [18] with a set of efficient global constraint propagators; this significantly reduces the encoding size of the problem. Two of the proposed propagators are based on depth-first search, and a third is a checker that analyses the strongly connected components (SCCs) of the game graph. These propagators filter edges that would close winning cycles for a selected opponent player and generate explanations for each filtering step, enabling their integration into a Lazy Clause Generation solver [28].

We show that our implementation not only outperforms existing SAT and CP-based approaches but also achieves performance comparable to that of state-of-the-art specialized non-CP-based algorithms, often matching or exceeding their results. This work contributes to establishing the CP framework as a practical tool for solving both general and specialized classes of parity games.

The remainder of this paper is organized as follows. We first present the theoretical foundations and necessary definitions used in further sections, and then introduce our proposed approach – including the design and implementation of the No-Opponent-Cycle propagators. After that, we provide different experimental results, a comparative analysis against state-of-the-art solvers, and present the extension of our method with additional constraints. At the end of the paper, we focus on the presentation of a number of concluding remarks, a discussion of our main findings, and some proposals for potential future research directions.

2 Preliminaries and Related work

This section introduces parity games and overviews the existing translations into Boolean Satisfiability (SAT) and Satisfiability Modulo Theories (SMT).

2.1 Parity Games

Parity games are two-player zero-sum games played on a labeled directed graph, called the *arena*, where two players, *Even* and *Odd* (denoted ○ or 0, and □ or 1, respectively), move a token along adjacent vertices depending on which player

controls the current vertex. This process generates a path of states visited by the token. In a parity game, each path is winning for either Player ○ or Player □. The goal of each player is to enforce a winning path in their favour. We now formalize parity games, including the definition of when a given path in the graph is winning for either player.

Definition 1 ([7]). *An arena G is a finite directed graph $G = (V = V_{\bigcirc} \cup V_{\square}, E, \Omega)$, where: V is a finite set of vertices, partitioned into $V_{\bigcirc}$ (controlled by Player ○) and $V_{\square}$ (controlled by Player □); $E \subseteq V \times V$ is a set of directed edges such that every vertex has at least one outgoing edge; and $\Omega : V \to \mathbb{N}$ is a labelling function that assigns to each vertex a non-negative integer, called its priority. For any vertex $v \in V$, we write $vE = \{w \in V \mid (v, w) \in E\}$ for its set of successors (adjacent to outgoing edges from v), and $Ev = \{u \in V \mid (u, v) \in E\}$ for its set of predecessors (adjacent to incoming edges to v).*

The game starts with a token at an initial vertex $v_0 \in V$. Player Even is responsible for moving the token when it is on a vertex in $V_{\bigcirc}$, and Player Odd is responsible for moving the token when it is on a vertex in $V_{\square}$. Because every node is required to have at least one outgoing edge, the token travels along the graph generating a path of infinite length. The winner is determined by the highest priority that is visited by the token infinitely often. Figure 1 illustrates an instance of a parity game arena.

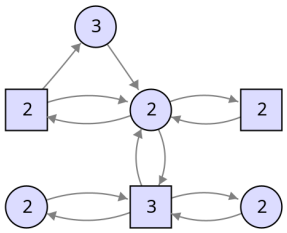


Fig. 1: A parity game with 7 vertices and 12 edges. Each vertex carries a priority from the set $\{2, 3\}$. Vertices controlled by Player ○ are shown as circles, while vertices controlled by Player □ are shown as squares.

Strategies, Plays, and Winning Conditions

A memoryless or *positional strategy* for player Even (player Odd) is a function $f_{\bigcirc} : V_{\bigcirc} \to V$ ($f_{\square} : V_{\square} \to V$) that dictates how the player should move the token when it is in one of the vertices the player controls. Every pair of memoryless strategies, one for each player, uniquely determines an infinite path π, called a *play*, in the game. Accordingly, $\Omega\pi$ is an infinite sequence of priorities. Let π_k be the $(k + 1)$th element in such a sequence of vertices, that is, in such a path. Because arenas are finite, such an infinite sequence of priorities has a (possibly empty) finite prefix $\Omega(\pi_0)\Omega(\pi_1)\ldots\Omega(\pi_h)$ of length h and an infinite

tail $\Omega(\pi_{h+1})\Omega(\pi_{h+2})\ldots$ of priorities that are seen infinitely often (given that the graph is finite and the strategies are memoryless/positional). We say that a play π is *winning* for player $\bigcirc$ (player $\Box$) if the highest priority that appears infinitely often in its associated sequence of priorities $\Omega\pi$ is even (odd).

Given an initial vertex v_0, a strategy is said to be winning from v_0 for player $\bigcirc$ (player $\Box$) if it only generates winning plays for player $\bigcirc$ (player $\Box$) starting at v_0 for all strategies of the other player. It is known that for every vertex $v \in V$ in a parity game, one of the two players always has a memoryless winning strategy from that vertex v. Solving a parity game means deciding, for every vertex $v \in V$, which player has a memoryless winning strategy from that vertex.

2.2 Solving parity games using SAT-Modulo-Theories

Besides dedicated methods for parity games, algorithms have been developed to *encode* a given game into constraint-based formalisms, in order to apply off-the-shelf solvers. One such approach [18] uses Satisfiability Modulo Theories (SMT) solvers to do so. That encoding is based on the following observation: A game is won by player $\bigcirc$ if, starting at v_0, every $v \in V_\bigcirc$ has at least one outgoing edge leading to an infinite path where the highest priority is even. Furthermore, every $v \in V_\Box$ must have all of its outgoing edges leading to infinite paths with the same condition. The idea, then, is to construct a CNF formula that is satisfiable if and only if the above condition for winning holds. Determining satisfiability therefore is equivalent to proving that Player $\bigcirc$ has a winning strategy, while unsatisfiability proves the same but for Player $\Box$. This approach introduces Boolean variables for the vertices and edges of the arena: S_v for every $v \in V$ and $T_{v,w}$ for every $(v, w) \in E$. Variables S_v and $T_{v,w}$ represent whether the corresponding vertices and edges are on a winning path for $\bigcirc$. A CNF formula can then capture the above simple path conditions for winning a game in the following way:

- $\Phi_\exists = \bigwedge_{v \in V_\bigcirc} (S_v \rightarrow \bigvee_{w \in vE} T_{(v,w)})$
- $\Phi_\forall = \bigwedge_{v \in V_\Box} (S_v \rightarrow \bigwedge_{w \in vE} T_{(v,w)})$
- $\Phi_V = \bigwedge_{w \in V, w \neq v_0} \left((\bigvee_{v \in Ev} T_{(v,w)}) \rightarrow S_w \right)$

These constraints state that for all vertices controlled by player $\bigcirc$, if a vertex is on the winning path, then at least one outgoing edge must be on the winning path ($\Phi_\exists$); for all vertices controlled by player $\Box$, if the vertex is on the winning path (for $\bigcirc$), then all outgoing edges must be on the winning path ($\Phi_\forall$); and for all vertices, if an edge leading to the vertex is on the winning path for player $\bigcirc$, then the vertex itself is on the winning path (Φ_V).

The final condition is slightly more difficult to implement: every *infinite* path must satisfy the parity condition, that is, the highest priority that occurs infinitely often along the path must be even. This can be locally encoded using a *progress measure* [18,20] expressed with additional integer variables. A variable x_p^v is introduced for every $v \in V$ and every $p \in \mathit{Odd}(G)$, corresponding to the set of all odd priorities in the game, defined as $\mathit{Odd}(G) = \{p \in \mathbb{N} \mid p \text{ is odd and } \Omega(v) = p, \text{ for some } v \in V\}$. These variables track *progress* along

every $t \in \mathit{Inf}(\pi)$, the set of priorities that appear infinitely often in π, ensuring that $x_p^v > x_p^w$ if the priority is odd, or $x_p^v \geq x_p^w$ if the priority is even. Since odd priorities require a strictly increasing chain of integers, cycles with odd priorities are ruled out by this constraint. On the other hand, for even priorities, only a non-strict inequality is required. The definition of the progress measure constraints is as follows ([18,12]):

- $\Phi_A = \bigwedge_{(v,w)\in E} \left(T_{(v,w)} \rightarrow \Psi_{(v,w)}\right)$
- $\Psi_{(v,w)} = \bigwedge_{p\in Odd^{>\Omega(w)}(G)}(x_p^v \geqslant x_p^w)$, *if* $\Omega(w)$ *is even*
- $\Psi_{(v,w)} = \bigwedge_{p\in Odd^{>\Omega(w)}(G)}(x_p^v \geqslant x_p^w) \wedge (x_{\Omega(w)}^v > x_{\Omega(w)}^w)$, *if* $\Omega(w)$ *is odd*

These constraints are instances of *difference logic*, which is supported by some SMT solvers. Thus, an SMT solver can determine the satisfiability of $(S_{v_0} \wedge \Phi_\exists \wedge \Phi_\forall \wedge \Phi_V \wedge \Phi_A)$, and, as such, decide whether player $\bigcirc$ has a winning strategy.

The difference logic constraints of the progress measure can also be encoded into pure CNF, which enables the use of modern SAT solvers to decide the winner of a parity game. This was the approach chosen in the paper that introduced the SMT encoding [18].

3 A Constraint-Based Approach Using an Efficient No-Opponent-Cycle Propagator

In this section, we present our approach to solving parity games by encoding them into a Constraint Satisfaction Problem (CSP) with three specialised propagators for the winning condition. Our method closely follows the framework presented in the previous section, determining satisfiability by evaluating if a game, starting at v_0, is won by player $\bigcirc$.

However, while we retain the constraints $\Phi_\exists$, $\Phi_\forall$, and Φ_V to ensure connectivity within the arena, we observe that the Φ_A constraints—based on progress measures—significantly hinder the scalability of SAT-based approaches. In the worst case, the size of the encoding grows cubically with the size of the arena. In practice, although the encoding is not always cubic, it is often still prohibitively large. For example, a Steady Game with 5,000 nodes may generate more than 215 million clauses. Such prohibitive instances are common in practice, even when the set $\mathit{Odd}(G)$ is small.

Our approach, then, replaces the progress measure with a global constraint that avoids the use of the integer variables x_p^v. We also define our model to enable search from both players' perspectives, aiming to determine, efficiently, if the instance is unsatisfiable for either Player $\bigcirc$ or Player $\Box$.

3.1 Characterising winning strategies globally

We first translate the game into a CSP. Thus, let $\mathsf{p} \in \{\bigcirc, \Box\}$ be a player and $\overline{\mathsf{p}} \in \{\bigcirc, \Box\}$, with $\overline{\mathsf{p}} \neq \mathsf{p}$, be its opponent.

Definition 2. *The CSP encoding of the parity game with arena $G = (V, E, \Omega)$, starting vertex v_0 and searching for the perspective of player p is defined as $\mathcal{CSP} = (\mathcal{X}, \mathcal{D}, \mathcal{C})$, where the set of variables is given by $\mathcal{X} = (\mathcal{V}_v)_{v \in V} \cup (\mathcal{E}_e)_{e \in E}$. All of these variables are Boolean, thus, $\mathcal{D} = \{\bot, \top\}$. The set of constraints, denoted by C, is defined as:*

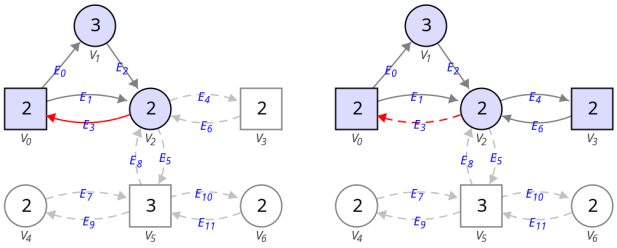
- $\mathcal{C}_{v_0} : \mathcal{V}_{v_0} = \top$
- $\mathcal{C}_{\mathsf{p}} : \bigwedge_{v \in V_{\mathsf{p}}} (\mathcal{V}_v \rightarrow (\sum_{w \in vE} \mathcal{E}_{(v,w)} = 1))$
- $\mathcal{C}_{\overline{\mathsf{p}}} : \bigwedge_{v \in V_{\overline{\mathsf{p}}}} (\mathcal{V}_v \rightarrow \bigwedge_{w \in vE} \mathcal{E}_{(v,w)})$
- $\mathcal{C}_E : \bigwedge_{w \in V, w \neq v_0} ((\bigvee_{v \in Ew} \mathcal{E}_{(v,w)}) \rightarrow \mathcal{V}_w)$
- $\mathcal{C}_{NOC} : \bigwedge_{v_0,\ldots,v_k} ((\bigwedge_{i \in 0 \ldots k-1} \mathcal{E}_{(v_i, v_{i+1})}) \rightarrow \bigwedge_{j \in 0 \ldots k-1}$
  $(v_j = v_k \rightarrow \max\{\Omega(v_i) \mid i \in j \ldots k\} \bmod 2 \neq \overline{\mathsf{p}}))$

Constraints $\mathcal{C}_{v_0}$, $\mathcal{C}_{\overline{\mathsf{p}}}$ and $\mathcal{C}_E$ directly correspond to the S_{v_0}, $\Phi_\forall$ and Φ_V constraints in the SMT encoding. We have adapted the $\Phi_\exists$ constraint into $\mathcal{C}_{\mathsf{p}}$ to enforce that *exactly one* outgoing edge must be activated for each vertex of player p. Additionally, we replace the progress measure constraint Φ_A with a global constraint $\mathcal{C}_{NOC}$, where NOC stands for No-Opponent-Cycle. This constraint holds iff for every path $v_0, \ldots, v_k$ where $1 \leq k \leq |V|$, if a cycle is formed by $v_j = v_k$ (for any $j < k$), then the parity of the maximum priority value on that cycle must differ from the $\overline{\mathsf{p}}$ value. This constraint forbids any cycle in which the highest priority that occurs infinitely often has the same parity as the opponent. Unlike previous formulations that assumed player $\Box$ as the default opponent, our construction generalizes to any arbitrary pair of players p and $\overline{\mathsf{p}}$. This ensures that player p is guaranteed to win if, after ruling out all possible winning strategies for $\overline{\mathsf{p}}$, the CSP remains satisfiable with p as the winner. Let us illustrate the filtering we want to achieve using this global constraint with an example.

Example 1. Let us update the game in Figure 1. Figure 2 shows the same arena, but with additional labels for the edges and vertices. Searching from the perspective of Player $\mathsf{p} = \bigcirc$ and starting at vertex V_0, after enforcing constraints $\mathcal{C}_{v_0}, \mathcal{C}_{\mathsf{p}}, \mathcal{C}_{\overline{\mathsf{p}}}, \mathcal{C}_E$ from Definition 2, assume that the solver makes an initial assignment of $\mathcal{V}_0 = \mathcal{V}_1 = \mathcal{V}_2 = \mathcal{E}_0 = \mathcal{E}_1 = \mathcal{E}_2 = \mathcal{E}_3 = \top$ (see Figure 2a), while the remaining variables are unassigned. At this point, Player $\Box$ has a winning strategy because edges $\mathcal{E}_0$, $\mathcal{E}_2$ and $\mathcal{E}_3$ form a cycle where the maximum priority occurring infinitely often is 3.

A checker of $\mathcal{C}_{NOC}$ would, in this situation, report unsatisfiability of the problem, triggering a backtrack and a search for an alternative assignment. The solver would continue the search, possibly exploring another assignment, $\mathcal{V}_0 = \mathcal{V}_1 = \mathcal{V}_2 = \mathcal{V}_3 = \mathcal{E}_0 = \mathcal{E}_2 = \mathcal{E}_4 = \mathcal{E}_6 = \top$ (see Figure 2b). Here, another cycle is formed, but the maximum priority occurring infinitely often is 2, which satisfies all constraints, making the problem satisfiable. As a result, starting from V_0, this game is won by Player $\bigcirc$.

Stronger versions of the $\mathcal{C}_{NOC}$ constraint can filter out assignments that necessarily lead to cycles with an odd maximum priority. For instance, if the former assignment had been considered first, the constraint could have determined in advance that $\mathcal{E}_3$ *must* be $\bot$, since otherwise an odd cycle would be present.



(a) Initial assignment (b) Satisfiable assignment

Fig. 2: A parity game with vertices $\{0..6\}$, edges $\{0..11\}$, and priorities $\{2, 3\}$.

3.2 A No-Opponent-Cycle checker

To enforce $\mathcal{C}_{NOC}$, we propose a propagator based on recursively decomposing the graph into Strongly Connected Components (SCCs). By definition, all vertices inside an SCC of size greater than 1 are part of at least one cycle, while single-vertex SCCs may still contain self-loops. Each resulting subgraph is then evaluated: if it contains a cycle in which the highest priority occurring infinitely often favors $\overline{\mathsf{p}}$, the current assignment is rejected. Otherwise, the maximum priorities favouring p are removed from the SCC, and new SCCs are computed on the reduced graph. If, in the end, all remaining single vertices lack both self-loops and $\overline{\mathsf{p}}$ priorities, the assignment is accepted.

Algorithm 1 ***NOCChecker***

Input: G
Output: $\top$ is valid; $\bot$ otherwise

```
1: for scc ∈ Tarjan(G) do
2:   if |scc| = 1 then
3:     v ← the unique vertex in scc
4:     if priority(v) mod 2 = p̄ ∧ vEv then return ⊥
5:   else
6:     bestSet ← bestVertices(scc)
7:     v ← an arbitrary vertex in bestSet
8:     if priority(v) mod 2 = p̄ then return ⊥
9:     NOCChecker(G \ bestSet)
10: return ⊤
```

Algorithm 1 is implemented recursively, with an argument corresponding to a subgraph induced by the vertices assigned $\top$. Before invoking the function, all variables are checked to ensure they are fixed. The function is invoked as $NOCChecker([\ v \mid \mathcal{V}[v] = \top\ ])$. The current subgraph G is decomposed into SCCs using Tarjan's algorithm [29], so that each SCC is analyzed independently (line 1). In each iteration, the algorithm checks whether the current SCC consists of a single vertex (line 2). If so, it verifies whether the vertex has a self-loop and whether its priority matches that of $\overline{\mathsf{p}}$; in such a case, the assignment is rejected (line 4). If the SCC contains more than one vertex, the algorithm identifies the set of vertices with the highest priority (line 6) and selects one arbitrarily to

check its parity. If the selected vertex's priority corresponds to $\overline{\mathsf{p}}$, the assignment is also rejected (line 8). Otherwise, the vertices in this highest-priority set are removed from the SCC, and the function is recursively called on the resulting subgraph (line 9).

The complexity of the NOC-Checker propagator is polynomial in the size of the input graph. Each call to Tarjan's algorithm computes the SCCs in linear time, i.e., $\mathcal{O}(|V| + |E|)$. In the worst case, the algorithm may recursively remove and reprocess subsets of vertices with maximal priority multiple times. However, each vertex can only be part of an attracting set and removed once per recursive level, ensuring that the total number of Tarjan calls remains linear in the number of vertices. As a result, the overall time complexity of the propagator remains $\mathcal{O}(|V| \cdot (|V| + |E|))$ in the worst case.

3.3 A No-Opponent-Cycle filter

In addition to NOC-Checker, we also propose a complete propagator based on Depth-first Search (DFS) to explore the arena. The resulting DFS tree captures all possible paths from v_0, and can be used to detect all existing cycles. We define the propagator in the context of a *Lazy Clause Generation* solver [28], to take advantage of the strong performance of this type of solver, in particular for problems that contain a large number of Boolean clauses. Algorithm 2 presents this version of the propagator, which checks for cycles and reports failure if a cycle with highest priority in favour of $\overline{\mathsf{p}}$ is detected.

Algorithm 2 ***NOCEager***

Input: $path_V$, $path_E$, v, e, $\mathcal{E}$, *defined*
Output: $\top$ is valid; $\bot$ otherwise

```
 1: if v ∈ path_V then
 2:     i_v ← index(v, path_V)
 3:     m ← max{f(v') | v' ∈ path_V[i_v :]}
 4:     if m mod 2 = p̄ then
 5:         reason ← clausify(¬path_E[i_v :])
 6:         if not (E[e] ← (⊥, reason)) then return ⊥
 7: else if defined then
 8:     for e' ∈ edges(v) where E[e'] ≠ ⊥ do
 9:         p_V ← path_V ∪ {v}
10:         p_E ← path_E ∪ {e'}
11:         w ← target(e')
12:         def ← E[e'] = ⊤
13:         NOCEager( p_V, p_E, w, e', E, def )
14: return ⊤
```

Algorithm 2 is also written in a recursive style. The arguments $path_V$ and $path_E$ represent the currently explored path in the DFS procedure, expressed in terms of both vertices and edges. The parameter v denotes the current vertex, e is the currently explored edge, and $\mathcal{E}$ is the array of Boolean variables associated with the edges of the graph. The No-Opponent-Cycle propagator invokes this

function as $NOCEager(\emptyset,\ \emptyset,\ start,\ -1,\ \mathcal{E},\ \top)$, where –1 indicates that no edge has been selected yet from $\mathcal{E}$, as valid edges are indexed with $e \geq 0$.

First, consider the case where the current vertex v is not already in the current path (i.e., the else case in line 7). This indicates that no cycle has been detected so far, and the algorithm recursively explores each outgoing edge from v whose value is not $\bot$, extending the path accordingly. A basic implementation of NOC-Eager would only explore edges for which $\mathcal{E}[e'] = \top$, thus limiting the search to assigned edges. However, the goal of this propagator is to allow the exploration of at most one unassigned edge, in order to proactively eliminate potential cycles before they become committed.

In contrast, if the current vertex v has appeared in the path (line 1), this implies that the current edge closes a cycle. The algorithm then determines the highest priority among the vertices in the cycle, and if this priority favors $\overline{\mathsf{p}}$ (line 4), the current edge is set to $\bot$. If the edge had already been assigned $\top$, this causes a conflict and triggers backtracking.

Since Lazy Clause Generation (LCG) solvers require a justification for each propagation step (to enable clause learning), all edges in the detected cycle are collected to form a clause stating that at least one must be assigned $\bot$ (line 5). For instance, in the scenario shown in Figure 2b, the clause $(\neg\mathcal{E}_0 \vee \neg\mathcal{E}_2 \vee \neg\mathcal{E}_4 \vee \neg\mathcal{E}_6 \vee \neg\mathcal{E}_3)$ would be added to prevent the opponent cycle.

The complexity NOC-Eager is primarily driven by the DFS traversal over the graph, with additional work performed when a cycle is detected. In the worst case, the algorithm explores every simple path in the graph starting from the initial vertex. This may result in an exponential number of recursive calls in the number of vertices since each branching decision adds a new path to explore. However, in practice, the number of recursive calls is bounded by the number of edges assigned to $\top$ and at most one unassigned edge, significantly reducing the branching factor. Each recursive call performs a constant number of operations, except when a cycle is found (line 1). Nevertheless, because the algorithm may visit multiple paths and cycles, and because each edge can potentially trigger multiple clause generation steps before being fixed, the overall worst-case complexity remains exponential in the size of the graph. This trade-off is justified by the early pruning capabilities enabled by detecting potentially winning opponent cycles before they are fully formed, especially in graphs with many unassigned edges or nondeterminism. In typical solver applications, where assignments propagate quickly and many edges are fixed early, the practical runtime of the propagator is significantly better than the theoretical worst case—as demonstrated by the experimental results presented in section 4, where this approach consistently outperforms alternatives in both speed and effectiveness.

3.4 Improved Filtering Strategy for the No-Opponent-Cycle Propagator

In order to mitigate the exponential behavior of NOC-Eager, we have developed a variant that memorizes cycles and can quickly remove some new cycles. To do so, one can use Algorithm 3, which incorporates a parameter *memo* that stores,

for each edge, the starting node of an infinite path and the associated highest priority. This information is updated whenever a cycle is found (line 4). If *memo* does not contain information about the next edge e', or a new highest priority was found, then the propagator proceeds recursively as before (lines 16 and 21). Otherwise, it uses the stored information for the new edge (line 4). This results in polynomial worst-case runtime, since the recursion is now bounded by the number of edges and the number of different priorities. However, the memoization only captures cycles with the same starting node. The propagator may therefore miss some cycles, rendering it incomplete. To preserve completeness, this variant is therefore used in conjunction with the NOC-Checker introduced earlier.

For the example of Figure 2, the updated version of No-Opponent-Cycle stores a tuple $(v = 5, m = 3)$ on edge E_5, indicating that the best priority reachable from this edge is 3 and that the infinite path starts at V_5. This prevents the exploration of the subgraph $\{V_4, V_5, V_6\}$ in future stages. Remembering cycles is particularly effective in games where the number of cycles favoring p exceeds those favoring $\overline{\mathsf{p}}$, as demonstrated below in the section on experiments.

Algorithm 3 ***NOCMemo***

Input: $path_V, path_E, v, e, \mathcal{E}$, *defined*, *memo*
Output: $\top$ is valid; $\bot$ otherwise

```
1: if v ∈ path_V then
2:    i_v ← index(v, path_V)
3:    m ← max{f(v') | v' ∈ path_V[i_v :]}
4:    memo[e] ← (v, m)
5:    if m mod 2 = p̄ then
6:       reason ← clausify(¬path_E[i_v :])
7:       if not (E[e] ← (⊥, reason)) then return ⊥
8: else if defined then
9:    for e' ∈ edges(v) where E[e'] ≠ ⊥ do
10:      (v_memo, m_memo) ← memo[e']
11:      p_V ← path_V ∪ {v}
12:      p_E ← path_E ∪ {e'}
13:      w ← target(e')
14:      def ← E[e'] = ⊤
15:      if (v_v, m_memo) = (−1, −1) then
16:         NOCMemo(p_V, p_E, w, e', E, def, memo)
17:      else
18:         i_v ← index(v_memo, path_V)
19:         m = max{f(v') | v' ∈ path_V[i_v :]}
20:         if m > m_memo then
21:            NOCMemo(p_V, p_E, w, e', E, def, memo)
22:         else
23:            memo[e] ← memo[e']
24: return ⊤
```

The three versions of the No-Opponent-Cycle propagator can be used to solve parity games, with NOC-Memo requiring the NOC-Checker.

3.5 Solving for p and $\overline{p}$ in Parallel

As our experiments will demonstrate, solver performance is heavily dependent on the game's underlying structure. A key observation is the asymmetry in computational cost inherent in game structure: properties that challenge p's solution often simplify $\overline{p}$'s, and vice versa. This problem duality allows us to transform and simplify the solution. By exploiting this, we can avoid a worst-case scenario for one player by solving the game from the opposite perspective. Consequently, solving the game from both perspectives in parallel significantly improves overall efficiency. This acceleration is largely attributed to the No-Opponent-Cycle propagator's capacity to rapidly prune the search space by eliminating $\overline{p}$'s winning options, effectively proving p has no winning strategy. We execute both perspectives concurrently and utilize the result obtained by the faster computation.

As confirmed by additional experiments (available on the Online Appendix), we confirm that state-of-the-art methods cannot exploit this duality, demonstrating consistent performance regardless of which player wins.

4 Experiments and Evaluation

To evaluate our approach, we implemented the No-Opponent-Cycle propagators using the Constraint Programming (CP) solver Chuffed (version 0.12.1) [11]. We utilized MiniZinc [27] to model additional constraints in selected experiments.

For comparison, we employed several external state-of-the-art solvers:

- SAT Solvers (Reduction SAT): We used **CaDiCaL** (v2.1.3) [4], **MiniSat** (v2.2.0) [14], **Kissat** (v4.0.2) [5], and zChaff (v2007.3.12) [26].
- State-of-the-Art (SOTA) Parity Game Solvers: We incorporated Oink (fresh GitHub version) [13], which includes several high-performance algorithms like Priority Promotion (**PP** and **PP**+) and Parys' improvements (**PAR**). We also included the Zielonka Recursive Algorithm (**ZRA**), with an improved attractor strategy as detailed in [1], which offers better perfonmance.

All experiments were conducted on an Intel Xeon 8260 CPU (24 cores, non-hyperthreaded) with 268.55 GB of RAM, running Linux. The complete source code, including implementations and experimental setups, is available at [19]. We evaluated all algorithms using a standard benchmark set commonly employed in parity game research, drawing from the PGSolver Library [17] and the Benchmarks for Parity Games [23]. The games were classified into four categories:

- Equivalence Checking (**EqC**): This group contains 77 games derived from formal equivalence checking problems, including strong bisimulation, weak bisimulation, and stuttering equivalence. These games are characterized by complex, structured connectivity patterns reflecting system design.
- Model Checking (**MoC**): This set consists of 106 games originating from temporal logic model checking (μ-calculus) against system models. These instances are considered organic and reflect real-world verification challenges.

- PGSolver Collection (**PGS**): This collection comprises 161 games sourced from the hardest known families of parity games, including ladder games, recursive ladder games, elevator verification, clique games, steady games, and Jurdziński games [2], likely to expose some of the worst performance.
- Random Games (**RaG**): This category includes 125 randomly generated games with varying sizes and priority distributions, designed primarily to test the general robustness and average-case performance of solving algorithms.

The benchmark sets were partitioned into two distinct size categories for evaluation, optimizing for the feasibility of the Reduction to SAT approach on smaller instances and focusing on scalability for larger instances:

Table 1: Time Performance of Algorithms on Small-Scale Instances, Average measured in seconds. (Max vertices: EqC/MoC $\approx 10^6$, PGS/RaG $\approx 10^3$).

Bench	**Oink**			**ZRA**	**NOC**		**Reduction SAT**		
	PP	PP+	PAR	ImpAttr	Eager	Memo	CaDiCal	Kissat	MiniSat
EqC	0.663	0.649	0.544	**0.190**	0.399	0.400	16.114	76.424	32.594
MoC	0.327	0.324	0.294	**0.172**	0.289	0.290	15.524	28.924	22.443
PGS	**0.126**	0.126	2.580	7.237	5.043	25.220	292.021	294.794	306.186
RaG	0.001	0.001	**0.001**	0.002	0.003	0.005	1.100	1.113	1.384

In Table 1, we compare the performance of the No-Opponent-Cycle propagators against two groups of competing approaches on small benchmark games: (1) other constraint-based methods that use Reduction to SAT, and (2) non-constraint-based methods, such as the ZRA algorithm (Improved Attractor) and the Oink algorithms (PP, PP+, PAR). The goal of this evaluation is to assess the effectiveness of the filtering techniques relative to existing SAT-based methods, which is the primary focus of this paper. Overall, No-Opponent-Cycle outperforms the SAT-based methods by up to three orders of magnitude. Furthermore, we observe that the Memoization approach (NOC-Memo) does not significantly improve performance over the Eager approach (NOC-Eager). Moreover, NOC-Eager achieves performance comparable to the best non-CP-based algorithms in practice: it is at most one order of magnitude slower on the hardest games (from the PGSolver Collection) and no more than a factor of two slower on all other instances. Additional comparative results with other CP methods are provided in the Online Appendix.

Table 2 presents the results for large games, where we evaluate the scalability of the No-Opponent-Cycle technique in comparison to ZRA and the Oink family of algorithms. SAT-based approaches are omitted, as they were unable to solve these larger instances within reasonable resource limits and therefore do not contribute to a meaningful scalability analysis. Overall, No-Opponent-Cycle shows strong and robust performance: in the Equivalence and Model Checking benchmarks, it is within a small constant factor of ZRA, which remains the fastest method, which is consistent with the main results reported in [1]. In the PGSolver Collection, No-Opponent-Cycle outperforms all Oink algorithms

Table 2: Time Performance of Algorithms on Large-Scale Instances, Average measured in seconds using PAR2 Scoring. (Max vertices: ≈ 35 Million).

Benchmark	**Oink**			**ZRA**	**NOC**	
	PP	PP+	PAR	ImpAttr	Eager	Memo
EqC	24.775	24.876	24.159	**14.490**	16.169	82.902
MoC	9.547	9.557	8.718	**3.929**	8.838	8.912
PGS	44.949	44.922	182.643	201.804	**32.189**	55.464
RaG	0.016	0.015	**0.015**	0.053	0.078	0.099

and also surpasses ZRA by a large margin. For Random Games, all algorithms solve the instances almost instantly, making performance differences negligible. These results demonstrate that No-Opponent-Cycle scales effectively and remains competitive with the best non-CP-based (dedicated) solvers on large games.

The runtime metrics for the No-Opponent-Cycle and ZRA approaches include both the solver execution time and the internal problem-preparation time (excluding I/O operations such as file reading). In contrast, for the SAT-based approaches we report only the solving time. This distinction is made because the preparation phase for the SAT-based methods—particularly the encoding generation—is substantially more expensive and was not a primary target of optimization in this work. Additional experiments comparing NOC-Eager, NOC-Memo, NOC-Checker with ZRA, together with their statistical analysis (AVG, SD, WR) presented in the Online Appendix, indicate that the NOC-Memo implementation does not provide a significant performance advantage over the NOC-Eager implementation on these benchmark instances.

4.1 Resource Utilization Analysis

A key motivation for developing the No-Opponent-Cycle propagator was to avoid the resource overhead associated with introducing the Boolean literals and clauses required for the progress measure variables and constraints. We also evaluated the memory consumption of No-Opponent-Cycle compared to other methods.

The results of these additional experiments are summarised in Figure 3. While No-Opponent-Cycle requires at most 700MB of memory games for 13 thousand vertices, and ZRA up to 17MB, the SAT-based approaches consume significantly more. In particular, we see that CaDiCaL, MiniSat, and Kissat report memory usage exceeding 39 GB for the same instances.

5 Games with Additional Constraints

As we will see in this section, our method flexibly integrates additional constraints—such as quantitative or structural requirements—without any modification to the core algorithm. This inherent adaptability to various constraints is a decisive advantage over non-CP approaches, which necessitate custom adaptations for every additional set of constraints in the problem.

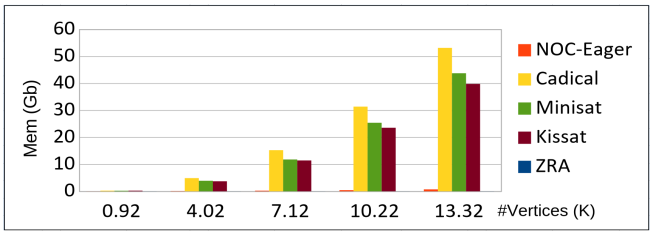


Fig. 3: Memory usage on Gb for solving Jurdziński parity games (one of the hardest games in the PGSolver collection), constrained to 13 thousand vertices.

5.1 Automated Synthesis of Reactive Systems

In (local) model checking, it suffices to determine the winner from a given vertex (satisfiable if won by Player ○, unsatisfiable if won by Player □). However, applications like automated synthesis [8,16,24] require explicit winning strategies, that is, a description of the decisions to be made by the winner at each vertex.

Our satisfiability encoding can capture multiple winning plays simultaneously. For instance, in Figure 2b, a solution assigning $\top$ to vertices $\mathcal{V}_0$ to $\mathcal{V}_3$ and edges $\mathcal{E}_0$, $\mathcal{E}_1$, $\mathcal{E}_2$, $\mathcal{E}_4$, and $\mathcal{E}_6$ captures two distinct plays. To extract individual winning plays, we extend the CSP with variables and constraints such that each solution corresponds to exactly one play. We define a sequence of integer variables $(\mathcal{P}_s)_{s\in\{0..|V|-1\}}$ to represent the path. More specifically, we also add:

- h: the position where the finite prefix ends;
- t: the start of the cycle (tail).

We then introduce three new constraints:

- $(\mathcal{C}_{path})$: $\mathsf{t} \in \{0..\mid V\mid -1\} \wedge \bigwedge_{s\in\{1..\mathsf{t}\}} \bigvee_{e=(v,w)\in E}(\mathcal{E}_e \wedge \mathcal{P}_{s-1} = v \wedge \mathcal{P}_s = w)$;
  This ensures each step follows a valid edge;
- $(\mathcal{C}_{cycle})$: $\mathsf{h} \in \{0..\mathsf{t}\} \wedge \mathcal{P}_{\mathsf{t}+1} = \mathcal{P}_{\mathsf{h}}$;
  It ensures the cycle closes at position $\mathsf{t}+1$;
- $(\mathcal{C}_{unique})$: $\bigwedge_{i\neq j\in\{0..\mathsf{t}\}}(\mathcal{P}_i \neq \mathcal{P}_j)$;
  This constraint enforces only one cycle.

These constraints can be added in MiniZinc without modifying the core algorithm of No-Opponent-Cycle. Using Chuffed as the backend, we enumerate all solutions to obtain the set of winning plays, represented as vertex sequences.

5.2 Energy Conditions/Games

We now consider games with additional quantitative objectives. In *Energy Games* [9], each edge carries an energy level (a weight), and a play is winning if the running sum of these levels remains non-negative at all times, in addition to satisfying the parity condition. This requirement can be encoded using the $\mathcal{P}$ variables introduced earlier. Given a set of energy levels $(L_e)_{e\in E}$, we define integer variables $(\mathcal{I}_s)_{s\in\{0..\mathsf{t}-\mathsf{h}+1\}}$ to represent the edges in the infinite cycle (tail) of the play:

– $\mathcal{C}_{infEdges} : \forall s \in \{0..\mathsf{t} - \mathsf{h} + 1\}(\mathcal{I}_s = E(\mathcal{P}_{s+\mathsf{h}}, \mathcal{P}_{s+\mathsf{h}+1}))$

We then impose the following constraint to ensure the total energy level in the cycle is non-negative:

– $\mathcal{C}_{energy} : \sum_{e \in \mathcal{I}} L_e \geq 0$

While a complete model would also enforce energy constraints over all finite prefixes (up to t), we omit these here for brevity. Together, these constraints suffice to model the energy condition, useful in several verification settings.

Many other extensions—-such as multi-dimensional energy bounds and other quantitative payoff constraints—-can be modelled similarly using additional constraints or objective functions [6,15,25,30]. Further work is needed to precisely define each requirement and establish the appropriate constraints.

6 Conclusions

We have introduced a novel constraint-based approach for parity games which does not require integer-based progress measures. The experiments show that our method significantly outperforms existing constraint-based approaches reducing runtime significantly, and improving on memory consumption by over 95% on large instances, achieving much better results in performance (time and space) compared to previous work, and is competitive against non-CP-based solvers.

Our key innovation, replacing a SAT encoding with a global constraint, not only improves performance but also provides a flexible framework that can be easily extended to accommodate additional constraints, and therefore more complex requirements. In this paper, we have shown that these improvements can enable practical verification of large-scale systems while maintaining the modularity and expressiveness of CP-based methods using a modern technology.

Our work goes beyond parity games by easily enabling new solutions to synthesis and providing the flexibility to adapt our core to solve multiplayer games, including more general games (e.g., ω-regular games [10]). Our results demonstrate that constraint-based approaches with carefully designed global constraints offer a promising direction for practical formal verification tools that balance expressiveness and computational efficiency.

References

1. Aggarwal, S., de la Banda, A.S., Yang, L., Gutierrez, J.: A matrix-based approach to parity games. In: Sankaranarayanan, S., Sharygina, N. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 666–683. Springer Nature Switzerland, Cham (2023)
2. Benerecetti, M., Dell'Erba, D., Mogavero, F.: Solving parity games via priority promotion. In: Chaudhuri, S., Farzan, A. (eds.) Computer Aided Verification. pp. 270–290. Springer International Publishing, Cham (2016)

3. Benerecetti, M., Dell'Erba, D., Mogavero, F., Schewe, S., Wojtczak, D.: Priority promotion with parysian flair. Journal of Computer and System Sciences **147**, 103580 (2025). https://doi.org/https://doi.org/10.1016/j.jcss.2024.103580, https://www.sciencedirect.com/science/article/pii/S0022000024000758
4. Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F.: CaDiCaL 2.0. In: Gurfinkel, A., Ganesh, V. (eds.) Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I. Lecture Notes in Computer Science, vol. 14681, pp. 133–152. Springer (2024). https://doi.org/10.1007/978-3-031-65627-9_7
5. Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F.: CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024. In: Heule, M., Iser, M., Järvisalo, M., Suda, M. (eds.) Proceedings of SAT Competition 2024: Solver, Benchmark and Proof Checker Descriptions. Department of Computer Science Report Series B, vol. B-2024-1, pp. 8–10. University of Helsinki, Finland (2024), https://cca.informatik.uni-freiburg.de/papers/BiereFallerFazekasFleuryFroleyksPollitt-SAT-Competition-2024-solvers.pdf
6. Bozzelli, L., Murano, A., Perelli, G., Sorrentino, L.: Hierarchical cost-parity games. Theoretical Computer Science **847**, 147–174 (2020). https://doi.org/https://doi.org/10.1016/j.tcs.2020.10.002, https://www.sciencedirect.com/science/article/pii/S030439752030565X
7. Brihaye, T., Bruyère, V., Goeminne, A., Thomasset, N.: On relevant equilibria in reachability games. Journal of Computer and System Sciences **119**, 211–230 (2021). https://doi.org/https://doi.org/10.1016/j.jcss.2021.02.009, https://www.sciencedirect.com/science/article/pii/S0022000021000246
8. Bruyère, V.: A game-theoretic approach for the synthesis of complex systems. In: Revolutions and Revelations in Computability, Lecture Notes in Computer Science, vol. 13359, pp. 52–63. Springer International Publishing AG, Switzerland (2022)
9. Chatterjee, K., Doyen, L.: Energy parity games. CoRR **abs/1001.5183** (2010), http://arxiv.org/abs/1001.5183
10. Chatterjee, K., Henzinger, T.A.: A survey of stochastic ω-regular games. Journal of computer and system sciences **78**(2), 394–413 (2012)
11. Chu, G., Stuckey, P.J.: Chuffed: A lazy clause generation constraint solver. GitHub repository (2018), https://github.com/chuffed/chuffed
12. Cotton, S., Asarin, E., Maler, O., Niebert, P., Yovine, S., Lakhnech, Y.: Some progress in satisfiability checking for difference logic. Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems pp. 263–276 (2004)
13. van Dijk, T.: Oink: An implementation and evaluation of modern parity game solvers. In: Beyer, D., Huisman, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 291–308. Springer International Publishing, Cham (2018)
14. Eén, N., Sörensson, N.: An extensible SAT-solver. In: Giunchiglia, E., Tacchella, A. (eds.) Theory and Applications of Satisfiability Testing (SAT 2003). Lecture Notes in Computer Science, vol. 2919, pp. 502–518. Springer (2003). https://doi.org/10.1007/978-3-540-24605-3_37
15. Fijalkow, N., Zimmermann, M.: Parity and Streett games with costs. Logical Methods in Computer Science **Volume 10, Issue 2** (Jun 2014). https://doi.org/10.2168/lmcs-10(2:14)2014, http://dx.doi.org/10.2168/LMCS-10(2:14)2014
16. Finkbeiner, B., Klein, F.: Reactive synthesis: Towards output-sensitive algorithms (2018), https://arxiv.org/abs/1803.10104

17. Friedmann, O., Lange, M.: The PGSolver Collection of Parity Game Solvers, Version 4.1 (June 2017), https://github.com/tcsprojects/pgsolver/blob/master/doc/pgsolver.pdf, accessed: 2025-03-13
18. Heljanko, K., Keinänen, M., Lange, M., Niemelä, I.: Solving parity games by a reduction to SAT. Journal of Computer and System Sciences **78**(2), 430–440 (2012). https://doi.org/https://doi.org/10.1016/j.jcss.2011.05.004, https://www.sciencedirect.com/science/article/pii/S0022000011000535, games in Verification
19. Hernandez, G., Garcia, J., Gutierrez, J., Tack, G.: Implementation and evaluation of no-opponent-cycle propagators for solving parity games. https://github.com/GonzaloHernandez/NoOpponentCycle (2025), version 1.0
20. Jurdziński, M.: Small progress measures for solving parity games. In: Reichel, H., Tison, S. (eds.) STACS 2000. pp. 290–301. Springer Berlin Heidelberg, Berlin, Heidelberg (2000)
21. Jurdziński, M.: Deciding the winner in parity games is in UP ∩ co-UP. Information processing letters **68**(3), 119–124 (1998)
22. Jurdziński, M., Morvan, R., Thejaswini, K.S.: Universal algorithms for parity games and nested fixpoints. In: Lecture notes in computer science, pp. 252–271. Lecture Notes in Computer Science, Springer Nature Switzerland, Cham (2022)
23. Keiren, J.J.A.: Benchmarks for parity games (extended version) (2015), https://arxiv.org/abs/1407.3121
24. Luttenberger, M., Meyer, P.J., Sickert, S.: Practical synthesis of reactive systems from LTL specifications via parity games: You can teach an old dog new tricks: making a classic approach structured, forward-explorative, and incremental. Acta informatica **57**(1-2), 3–36 (2020)
25. Mogavero, F., Murano, A., Sorrentino, L.: On promptness in parity games. Fundamenta Informaticae **139**(3), 277–305 (2015). https://doi.org/10.3233/FI-2015-1235, https://doi.org/10.3233/FI-2015-1235
26. Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S.: Chaff: Engineering an efficient SAT solver. In: Proceedings of the 38th Annual Design Automation Conference (DAC 2001). pp. 530–535. ACM (2001). https://doi.org/10.1145/378239.379017
27. Nethercote, N., Stuckey, P.J., Becket, R., Brand, S., Duck, G.J., Tack, G.: MiniZinc: Towards a standard CP modelling language. In: Bessiere, C. (ed.) Principles and Practice of Constraint Programming - CP 2007, 13th International Conference, CP 2007, Providence, RI, USA, September 23-27, 2007, Proceedings. Lecture Notes in Computer Science, vol. 4741, pp. 529–543. Springer (2007). https://doi.org/10.1007/978-3-540-74970-7_38, https://doi.org/10.1007/978-3-540-74970-7_38
28. Ohrimenko, O., Stuckey, P.J., Codish, M.: Propagation via lazy clause generation. Constraints An Int. J. **14**(3), 357–391 (2009). https://doi.org/10.1007/S10601-008-9064-X, https://doi.org/10.1007/s10601-008-9064-x
29. Tarjan, R.: Depth-first search and linear graph algorithms. SIAM journal on computing **1**(2), 146–160 (1972)
30. Weinert, A., Zimmermann, M.: Easy to win, hard to master: Optimal strategies in parity games with costs. In: Log. Methods Comput. Sci. (2016), https://api.semanticscholar.org/CorpusID:1611141
31. Zielonka, W.: Infinite games on finitely coloured graphs with applications to automata on infinite trees. Theoretical computer science **200**(1), 135–183 (1998)

Appendix A: Additional experiments

This experiment focuses on CP and SMT solvers. We have selected Jurdziński graphs for this experiment, which allows us to vary the size of the graphs. We selected graphs with 10 blocks and between 10 and 80 levels (in steps of 10).

Table 3: Solving time on the Jurdziński Graphs $J_{level,10}$ CP-CP. (*) Represent times exceeding 5 minutes.

d	**No-Opponent-Cycle**		**Other Solvers**		
	□ Eager	○ Memo	Mzn(Gecode)	Mzn(CP-SAT)	Z3
10	0.000	0.008	0.59	0.71	0.22
20	0.000	0.025	3.06	2.59	0.58
30	0.000	0.039	10.57	6.15	2.64
40	0.000	0.087	28.53	12.52	7.76
50	0.001	0.089	63.77	22.47	18.11
60	0.001	0.097	126.88	37.41	35.76
70	0.001	0.158	231.51	58.22	65.62
80	0.001	0.185	*	88.89	111.80

Table 3 presents the solving time in seconds of four approaches: No-Opponent-Cycle, a MiniZinc CP model (expressing the progress measure using integer variables and constraints) solved by Gecode and Google CP-SAT, and an SMT model using difference logic to implement the progress measure, solved using Z3. The results provide strong evidence that Chuffed with the No-Opponent-Cycle propagator clearly outperforms the other approaches by several orders of magnitude on these graphs. Furthermore, some solutions, such as MiniZinc with Gecode, used over 256 GB of memory to complete the task.

Table 4: Average solving times (in seconds) on special game classes, categorized by winner.

Game class	**Win**	**ZRA**	**NOC from ○**		**NOC from □**		**SAT Solvers**		
			Eager	Memo	Eager	Memo	CaDiCal	MiniSat	Kissat
RaG	○	0.009	0.040	0.800	0.003	0.004	10.393	16.069	14.936
RaG	□	0.008	0.004	0.004	0.035	0.776	9.216	6.189	13.369
StG	○	0.009	0.032	0.680	0.003	0.003	132.217	233.167	158.103
StG	□	0.009	0.002	0.003	0.028	0.627	62.470	29.582	32.775
ClG	○	0.010	0.142	1.725	0.001	0.002	20.548	57.217	22.362
ClG	□	0.026	0.005	0.006	0.472	5.632	22.856	8.702	11.572

In Table 4, we evaluated performance in three classes of arenas: RandGames (RaG), SteadyGames (StG), and ClickGames (ClG). For each class, we generated instances and solved them using the ZRA solver, NOC-Eager (differing by the player of interest), and three SAT-based solvers (CaDiCaL, MiniSat, and Kissat).

Table 4 summarizes the average solving times over 35 runs per configuration. Each run starts from a randomly chosen initial vertex v_0. Results are grouped by game class, with the second column indicating the winning player (○ or □) for each configuration. The remaining columns report the average solving times (in seconds) for each solver and variant.

In all instances, every variant of No-Opponent-Cycle outperforms the SAT solvers by at least three orders of magnitude. Moreover, one of the No-Opponent-Cycle variants achieves performance comparable to ZRA, which is the best performing non-CP-based procedure in practice [1]. Notably, when we select p as the player of interest, No-Opponent-Cycle tends to be faster in cases where $\overline{\mathsf{p}}$ wins. This is because the algorithm can often conclude early by detecting the unsatisfiability of the problem.

Table 5: Average solving times (in seconds) on the RandomGamesSet-500, grouped by game size and winner.

Game Size $\lvert V\rvert \times \lvert E\rvert$	**Win**	**ZRA**	**NOC from ○**			**NOC from □**		
			Checker	Eager	Memo	Checker	Eager	Memo
8K × 20	○	0.020	26.041	0.125	2.759	60.000	0.009	0.011
8K × 20	□	0.020	60.000	0.011	0.013	36.396	0.186	3.532
20K × 20	○	0.061	31.347	0.502	14.493	60.000	0.026	0.029
20K × 20	□	0.062	60.000	0.032	0.037	27.250	0.485	14.268
50K × 20	○	0.209	23.382	2.323	60.000	26.437	0.076	0.087
50K × 20	□	0.205	23.085	0.084	0.096	21.387	2.422	60.000
90K × 20	○	0.427	60.000	6.905	60.000	60.000	0.148	0.163
90K × 20	□	0.422	60.000	0.172	0.196	60.000	7.129	60.000
150K × 10	○	0.331	60.000	6.050	60.000	60.000	0.138	0.147
150K × 10	□	0.317	60.000	0.244	0.254	60.000	13.221	60.000

In Table 5 we compared the performance of No-Opponent-Cycle against ZRA using a benchmark of 500 randomly generated games of varying sizes and edge densities. The sizes range from 8,000 to 150,000 vertices, with a maximum of 10 or 20 outgoing edges per vertex. In each game, the initial vertex v_0 was also selected randomly Table 5 reports the average solving times (in seconds), grouped by game size and by the winning player (○ or □). Each row presents the performance of ZRA, along with two No-Opponent-Cycle variants from Player ○'s perspective (Eager, Memo) and two corresponding variants from Player □'s perspective. The results show that the NOC-Eager variant was consistently competitive with ZRA, often achieving similar or better solving times. To support these observations, we conducted a standard deviation analysis and a Wilcoxon signed-rank test, both of which provide strong statistical evidence. These results, along with the full version of the table, are provided in the appendix, which also highlights the differences between the No-Opponent-Cycle filters and the Checker variant; a timeout of 60 seconds was set on each solver run.

Table 6: Solving times on Jurdziński graphs $J_{d,10}$. Symbol (*) represents unsolvable by the solver. [20].

d	ZRA	No-Opponent-Cycle		SAT Solvers			
		□ *Eager*	○ Memo	CaDiCaL	MiniSat	Kissat	zChaff
10	0.009	*0.000*	0.008	0.130	0.292	1.040	0.156
30	0.140	*0.000*	0.039	1.700	3.073	8.670	0.670
50	0.564	*0.001*	0.089	5.680	9.329	15.550	1.558
70	1.451	*0.001*	0.158	12.940	19.726	26.320	2.930
90	2.975	*0.001*	0.240	93.150	34.565	41.140	*
110	5.309	*0.001*	0.344	134.840	55.368	62.930	
130	8.616	*0.001*	0.468	193.500	80.627	85.320	*
150	13.070	*0.002*	0.610	260.590	111.662	115.420	
170	18.826	*0.002*	0.769	342.000	148.768	147.460	*
190	26.131	*0.002*	0.956	437.110	192.320	182.210	
210	35.045	*0.002*	1.145	539.770	244.924	230.440	*
230	45.780	*0.002*	1.363	655.450	297.770	283.340	
250	58.552	*0.002*	1.599	749.670	368.385	341.040	*
270	73.619	*0.003*	1.866	879.440	447.689	395.860	
290	90.890	*0.003*	2.127	1057.340	539.538	468.190	*
310	110.554	*0.003*	2.420	1207.110	623.698	540.090	
330	132.978	*0.003*	2.819	1403.980	717.835	585.110	*
350	158.433	*0.003*	3.197	1588.480	809.985	630.480	
370	187.527	*0.004*	3.562	1767.350	928.425	711.390	*
390	219.209	*0.004*	3.941	2113.110	1039.260	818.460	
410	254.540	*0.004*	4.348	2341.200	1157.990	897.660	*
430	292.778	*0.004*	4.742	2612.670	1387.010	991.170	

The Table 6 shows the set of experiments compares the No-Opponent-Cycle propagator with ZRA and SAT-based approaches, using three different solvers: CaDiCaL, MiniSat, and Kissat. For completeness, we included results for zChaff, as it was used in previous work [18], though it is no longer competitive. Table 6 shows solving times in seconds. In this experiment, we extended the range of levels (and thus the problem size) up to 410. The NOC-Memo propagator, when solving from player ○ 's perspective, consistently outperforms SAT-based approaches by at least two orders of magnitude and exceeds ZRA's performance by more than one order of magnitude. Moreover, the full version of the table, provided in the appendix, shows that when the problem is flipped to search from player □ 's perspective, NOC-Eager performs even faster, achieving speedups of over four orders of magnitude compared to ZRA on the largest instances.

All the results reported above omit initialization times. For example, SAT solvers required over 300 seconds to initialize on the largest instances, whereas No-Opponent-Cycle took less than 30 milliseconds in the same game. In contrast, ZRA does not involve a significant initialization phase.

Appendix B: Statistical Evaluation

Table 7: Statistical analysis based on Table 5 comparing ZRA and NOC-Eager from □'s perspective. The table includes mean (AVG) and standard deviation (SD) of solving times, along with Wilcoxon signed-rank test statistics (Samples, WR+, WR-).

Game Size $\lvert V\rvert \times \lvert E\rvert$	**Win**	**ZRA** AVG	SD	**NOC from □** AVG	SD	**Wilcoxon SR** Samples	WR+	WR-
8K × 20	○	0.020	0.001	0.009	0.001	84	0	-3570
20K × 20	○	0.061	0.061	0.026	0.002	55	0	-1540
50K × 20	○	0.209	0.025	0.076	0.006	44	0	-990
90K × 20	○	0.427	0.094	0.148	0.019	50	0	-1275
150K × 10	○	0.331	0.082	0.138	0.006	24	0	-520

Table 8: Statistical analysis based on Table 5 comparing ZRA and NOC-Eager from ○'s perspective. The table includes mean (AVG) and standard deviation (SD) of solving times, along with Wilcoxon signed-rank test statistics (Samples, WR+, WR-).

Game Size $\lvert V\rvert \times \lvert E\rvert$	**Win**	**ZRA** AVG	SD	**NOC from ○** AVG	SD	**Wilcoxon SR** Samples	WR+	WR-
8K × 20	□	0.020	0.001	0.011	0.004	74	75	-2700
20K × 20	□	0.062	0.004	0.032	0.014	45	10	-1025
50K × 20	□	0.205	0.028	0.084	0.018	56	0	-1596
90K × 20	□	0.422	0.100	0.172	0.052	46	1	-1080
150K × 10	□	0.317	0.040	0.244	0.148	22	33	-220

SOTA methods (ZRA) cannot exploit this duality; as shown in Tables 1–2, their performance remains consistent regardless of which player wins (we have also verified this for all original games and their complements)

The statistical evidence presented in Table 7 and Table 8, suggests that NOC-Eager achieves comparable performance to ZRA, with a consistent trend in its favor. In all cases, NOC-Eager records lower average solving times with smaller standard deviations, indicating slightly more stable behavior. The Wilcoxon signed-rank test supports this observation: the number of positive ranks (WR+) is generally low, and the negative rank sums (WR–) are consistently large, providing statistically significant—though not drastic—evidence that NOC-Eager tends to perform better across the tested instances.